

同步方法之三



同步方法的技术内幕

北京理工大学计算机学院
金旭亮

自己试验，了解技术内幕



自己写一个简单的同步方法，然后使用javap去查看javac编译器为它生成的字节码，会发现以下事实：

编译器会为“同步方法”添加一个ACC_SYNCHRONIZED标记，通知JVM，这是一个同步方法。

同步方法是基于锁实现的.....



Java平台中的所有对象都有唯一的一个与之关联的锁。这种锁被称为“内部锁 (Intrinsic Lock)”，这个锁保存在对象头（即内存中保存在对象开头的若干个字节）中，本质上就是几个二进制数位值，JVM规范对这些值代表锁的哪些信息进行了统一规范。



线程之间“争夺”锁的过程，主要体现为线程堆栈与对象头之间的数据复制及修改过程，相当复杂，这些细节是JVM实现者需要考虑的。

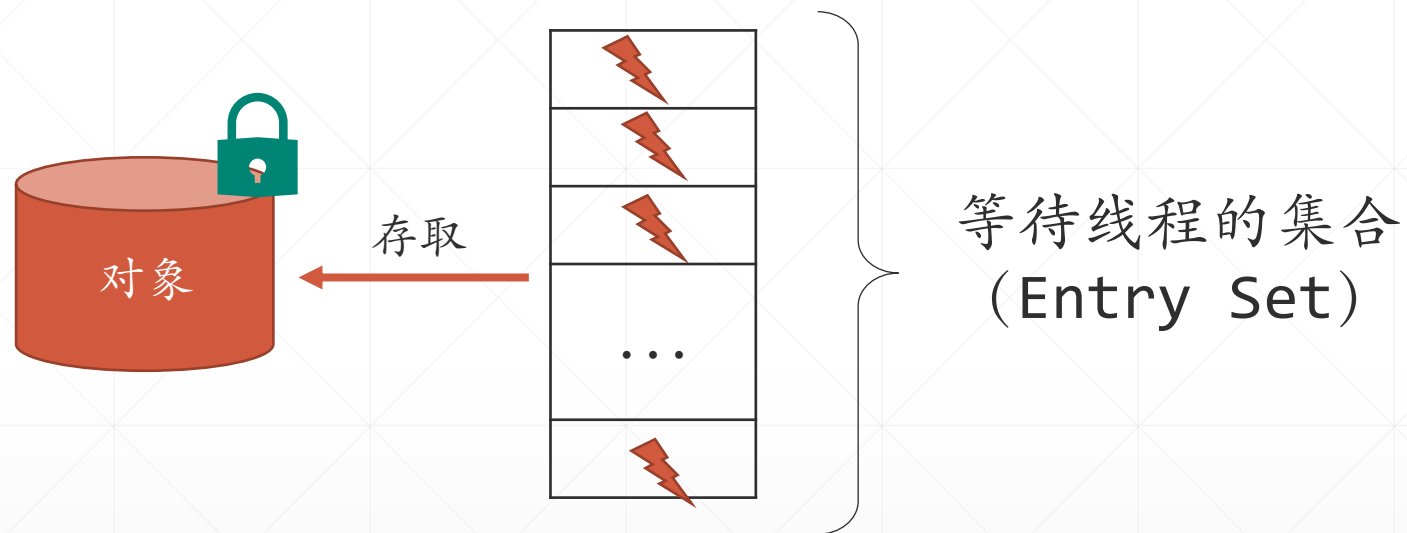


内部锁可以保障数据存取的原子性、数据修改可见性和指令的有序性。



同步方法使用的就是内部锁。

内部锁的调度



由于锁的存在，一次只能有一个线程独占访问此对象。

到底锁的是“啥”？



同步的实例方法，“锁定”的是单个对象实例。



同步的静态方法，“锁定”的是类型（即Class）实例。

关于Class对象



Class对象封装了特定类型的一些信息，诸如这个类型有哪些字段，有几个构造函数，这些构造函数有几个参数之类，它是“反射（Reflection）”技术的基础类型。



特定类（比如String）创建的所有对象，共享同一个类型（即Class）实例。



Object类中定义有以下方法，用于获取一个类型（Class）实例：

```
public final native Class<?> getClass();
```



注意区分“Class”和“class”，前者是一个“数据类型（Data Type）”，后者是一个语言关键字，用于定义一个类。

synchronized同步锁的基本原理

